

Laporan Praktikum Kontrol Cerdas

Nama : Muhammad Faizal Fachri
NIM : 224308092
Kelas : TKA 7D
Akun Github (Tautan) : github.com/muhammadfaizalfachri

1. Judul Laporan

Reinforcement Learning for Autonomus Control

2. Tujuan Percobaan

Tujuan dari percobaan kali ini, sebagai berikut:

1. Mahasiswa mampu memahami konsep dasar *Reinforcement Learning* (RL) dalam sistem kendali
2. Mahasiswa mampu mengimplementasikan agen RL menggunakan algoritma Deep Q-Network (DQN).
3. Mahasiswa mampu Menggunakan OpenAI Gym sebagai simulasi lingkungan untuk pelatihan RL.
4. Mahasiswa mampu Melatih dan menguji agen RL untuk mengontrol lingkungan secara otonom.

3. Landasan Teori

Reinforcement Learning (RL) adalah cabang dari pembelajaran mesin yang berfokus pada bagaimana agen cerdas harus mengambil tindakan dalam lingkungan dinamis untuk memaksimalkan sinyal imbalan. Berbeda dengan pembelajaran terawasi, RL tidak memerlukan pasangan *input-output* berlabel dan tidak memerlukan koreksi eksplisit terhadap tindakan suboptimal. Sebaliknya, fokusnya adalah menemukan keseimbangan antara eksplorasi (menjelajahi wilayah yang belum diketahui) dan eksploitasi (memanfaatkan pengetahuan yang ada) dengan tujuan memaksimalkan imbalan kumulatif (Andreanus & Kurniawan, 2018).

OpenAI Gym adalah toolkit sumber terbuka berbasis Python yang dirancang untuk mengembangkan dan membandingkan algoritma reinforcement learning dengan menyediakan antarmuka standar untuk

berkomunikasi antara algoritma pembelajaran dan lingkungan, serta seperangkat lingkungan standar yang sesuai dengan antarmuka tersebut (Brockman et al., 2016).

Deep Q-Network (DQN) adalah salah satu algoritma *Reinforcement Learning* berbasis Q-learning yang menggunakan jaringan saraf dalam (deep neural network) untuk memprediksi nilai Q (Q-value) dari setiap aksi yang mungkin diambil oleh agen dalam suatu keadaan tertentu (Mnih et al., 2015).

4. Analisis dan Diskusi

Analisis:

MountainCar-v0 merupakan sebuah lingkungan yang menantang, di mana sebuah mobil harus mencapai puncak bukit meskipun tidak memiliki tenaga yang cukup untuk langsung sampai ke tujuan. Lingkungan ini memiliki dua variabel keadaan, yaitu posisi dan kecepatan mobil, serta tiga pilihan aksi yang dapat diambil agen, yaitu bergerak ke kiri, diam, atau bergerak ke kanan. Setiap langkah yang dilakukan menghasilkan *reward* standar sebesar -1, sehingga agen terdorong untuk menemukan strategi tercepat mencapai puncak. Tantangan utama terletak pada kebutuhan agen untuk mengayunkan mobil maju-mundur guna memperoleh momentum sebelum akhirnya mampu menaklukkan tanjakan.

Untuk mengatasi masalah tersebut, digunakan Reinforcement Learning (RL) supaya agen bisa belajar membuat keputusan yang benar lewat interaksi dengan lingkungannya. Agen akan mencoba berbagai aksi, lalu mendapat balasan berupa reward jika tindakannya benar atau punishment jika salah, dan dari situ agen belajar menyusun strategi yang lebih baik. Karena kondisi lingkungan bersifat kontinu dan cukup rumit, algoritma Deep Q-Network (DQN) dipilih sebagai solusi. DQN menggunakan jaringan saraf untuk memperkirakan nilai Q, sehingga proses belajar jadi lebih efisien dan peluang agen mencapai tujuannya semakin besar

Namun, kinerja agen sangat dipengaruhi oleh parameter yang digunakan. Gamma yang tinggi mendorong agen untuk fokus pada reward jangka panjang, sedangkan gamma rendah membuat agen lebih

memprioritaskan reward yang cepat didapat. Epsilon yang tinggi pada awal pelatihan penting untuk eksplorasi, tetapi jika bertahan terlalu lama akan memperlambat proses belajar. Sementara itu, *learning rate* yang terlalu besar dapat menyebabkan ketidakstabilan, sedangkan learning rate yang terlalu kecil membuat proses konvergensi berlangsung lambat.

Selama proses pelatihan, ada beberapa tantangan yang muncul. Salah satunya adalah sulit menjaga keseimbangan antara mencoba hal baru (eksplorasi) dan memakai strategi yang sudah terbukti (eksploitasi). Selain itu, reward yang jarang muncul atau tidak jelas membuat agen kesulitan menilai apakah tindakannya sudah tepat atau belum. Pembelajaran juga bisa jadi tidak stabil, terutama saat memakai jaringan saraf yang rumit. Lingkungan dengan kondisi yang banyak dan kompleks semakin menambah kesulitan. Di sisi lain, ada risiko agen hanya jago di satu lingkungan saja (*overfitting*), tapi gagal saat diuji pada kondisi yang berbeda.

Diksusi:

1. Reinforcement Learning dapat diterapkan dalam sistem kendali nyata, misalnya untuk menggerakkan dan menyeimbangkan robot secara otomatis, mengatur kecepatan dan menghindari rintangan pada kendaraan otonom, mengoptimalkan penggunaan energi pada gedung pintar atau jaringan listrik, mengelola proses produksi dan penjadwalan mesin di manufaktur, serta mengendalikan parameter seperti suhu, tekanan, atau aliran bahan dalam proses industri.
2. Strategi eksplorasi dan eksploitasi pada agen RL dapat dioptimalkan dengan mengatur epsilon pada metode epsilon-greedy, di mana epsilon tinggi di awal agar agen banyak mencoba aksi baru, kemudian dikurangi agar lebih sering memilih aksi yang terbukti baik, serta dapat dibantu teknik lain seperti decay rate adaptif, softmax, atau UCB untuk menyeimbangkan eksplorasi dan eksploitasi sehingga pembelajaran lebih efisien.
3. *Reinforcement Learning* dapat diterapkan dalam berbagai sistem kendali nyata, seperti mengajarkan robot untuk bergerak atau menyeimbangkan secara otomatis, mengendalikan kendaraan otonom untuk mengatur kecepatan dan menghindari rintangan,

mengoptimalkan penggunaan energi pada gedung pintar atau jaringan listrik, mengatur proses produksi dan penjadwalan mesin di sistem manufaktur, serta mengendalikan parameter seperti suhu, tekanan, atau aliran bahan dalam proses kimia atau industri.

5. Assignment

Pada percobaan minggu keempat, eksperimen difokuskan pada implementasi algoritma Deep Q-Network (DQN) untuk melatih agen dalam environment MountainCar-v0, dengan tujuan agar agen dapat mencapai puncak bukit menggunakan aksi yang optimal. Disini saya menggunakan beberapa library, antara lain gymnasium untuk *environment* RL, numpy untuk operasi numerik, *random* untuk pemilihan acak, serta deque untuk menyimpan pengalaman agen. Model jaringan saraf dibangun menggunakan tensorflow.keras, sedangkan agen DQN diatur dengan parameter penting seperti gamma sebagai faktor diskon, epsilon untuk eksplorasi, dan *learning rate* untuk optimasi.

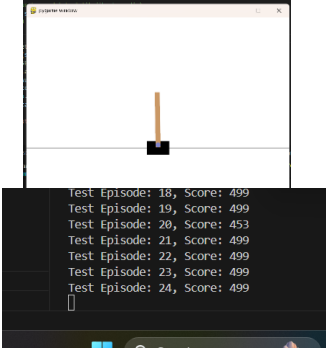
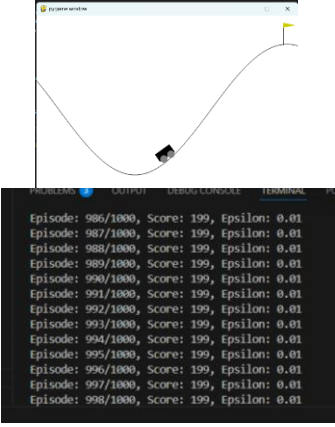
Pada percobaan di environment CartPole-V1, agen diminta untuk menjaga tiang tetap tegak di atas gerobak. Lingkungan ini tergolong sederhana, sehingga agen cepat belajar. Setiap kali agen berhasil menyeimbangkan tiang, reward positif diberikan. Pelatihan pada CartPole relatif singkat dan agen dapat mencapai performa stabil dalam beberapa ratus episode karena kondisi lingkungan mudah diprediksi.

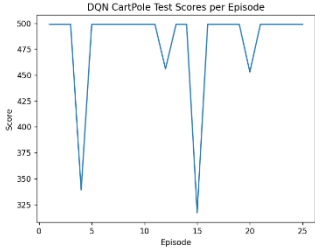
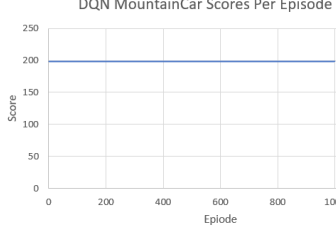
Pada MountainCar-v0, agen menghadapi tantangan lebih besar. Mobil tidak bisa langsung mencapai puncak bukit, sehingga agen harus belajar membangun momentum dengan bergerak maju dan mundur. *Reward* di sini lebih sulit diperoleh karena mobil sering gagal mencapai puncak, sehingga proses pembelajaran menjadi lebih lambat. Proses pelatihan membutuhkan lebih banyak episode sebelum agen dapat mencapai performa yang baik, dan nilai *reward* dapat berfluktuasi lebih besar dibandingkan CartPole.

Secara keseluruhan, eksperimen menunjukkan bahwa kompleksitas lingkungan sangat memengaruhi kecepatan belajar dan stabilitas agen, di mana lingkungan sederhana memungkinkan agen cepat mencapai tujuan,

sedangkan lingkungan yang lebih menantang memerlukan strategi belajar yang lebih matang dan proses pelatihan yang lebih panjang.

6. Data dan Output Hasil Pengamatan

No.	Kondisi	Hasil Pengamatan	
		ChartPole-v1	MountainCar-v0
1.	Waktu Pelatihan	CartPole-v1 lebih cepat mencapai performa optimal dalam waktu yang relatif lebih singkat	MountainCar-v0 relatif waktunya lebih lama
2.	Jumlah Episode Maksimum	CartPole-v1 dilatih hingga 1000-episode sudah cukup untuk mencapai skor maksimal	MountainCar-v0 dilatih 1000-10.000 episode agar agent dapat belajar mendarat dengan baik
3.	Kinerja Agen	Kinerja yang optimal dengan skor yang lebih tinggi	Ketika sedang melakukan pekerjaan susah untuk mencapai puncak
4.	Kondisi Berhasil	Episode berakhir jika tiang jatuh atau mencapai skor 500 (mana yang lebih dahulu tercapai)	Episode berakhir jika mobil mencapai puncak bukit (flag)
5.	Hasil Test Episode	 Pengujian dilakukan dengan 25 episode dengan rata-rata <i>score</i> 499	 Pada MountainCar score yang dihasilkan selama 1000 episode yaitu konstan 199

6.	Grafik	 Grafik menunjukan pengujian selama 25 episode terjadi penurunan score pada episode 15 dengan score 320	 Grafik menunjukan selama 1000 episode menghasilkan score terus-menerus 199 karena itu adalah nilai maksimal yang bisa dihasilkan
----	--------	---	---

7. Kesimpulan

Berdasarkan percobaan yang telah dilakukan maka dapat diambil kesimpulan sebagai berikut:

1. Semakin banyak jumlah episode yang dilakukan pada saat proses *train* maka score yang dihasilkan saat pengujian akan bagus juga.
2. Parameter penting seperti gamma, epsilon, dan *learning rate* sangat memengaruhi keberhasilan pelatihan.
3. Pada awal dilakukan latihan, performa agen rendah karena lebih banyak mencoba gerakan secara acak (eksplorasi).
4. Percobaan ini membuktikan bahwa *Reinforcement Learning* bisa diterapkan secara efektif untuk kontrol sederhana dan berpotensi dikembangkan ke sistem kendali nyata yang lebih kompleks.

8. Saran

Untuk membuat agen DQN bekerja lebih baik di lingkungan yang lebih sulit, kita bisa mencoba mengatur ulang parameter lewat berbagai percobaan, misalnya memakai dueling DQN, *prioritized experience replay*, atau menambah jumlah neuron dan lapisan pada jaringan. Untuk kasus seperti MountainCar, bisa juga dicoba metode lain seperti *actor-critic* atau *policy-based reinforcement learning* supaya agen lebih cepat belajar. Selain itu, kalau menggunakan hardware lebih kuat seperti GPU, proses latihan akan jauh lebih cepat, terutama di lingkungan yang rumit.

9. Daftar Pustaka

- Andreanus, J., & Kurniawan, A. (2018). Sejarah, Teori Dasar dan Penerapan Reinforcement Learning: Sebuah Tinjauan Pustaka. *Jurnal Telematika*, 12(2), 113–118. <https://doi.org/10.61769/telematika.v12i2.193>
- Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., & Zaremba, W. (2016). *OpenAI Gym*. 1–4. <http://arxiv.org/abs/1606.01540>
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533. <https://doi.org/10.1038/nature14236>